

Práctica 9 – Convenciones de Codificación

1. Objetivo

Aplicar estándares, convenciones y estilos de programación que contribuyan a la legibilidad, el mantenimiento y la reutilización del software, realizando revisiones de código en componentes de software utilizando una lista de verificación proporcionada..

Grupos: Individual.

Prerrequisito:

- Laboratorio 8
- Lenguaje de Programación: Java, C# o Python
- **IDE:** Eclipse, IntelliJ IDEA, VS Code MS Visual Studio, ...
- **Repositorio** en *GitHub*

2. Actividades: En la implementación de cada requisito/módulo/paquete/componente asignado por el líder del equipo, el miembro responsable debe:

- 1) Aplicar convenciones de codificación del lenguaje de programación escogido en la implementación bajo su responsabilidad.
- 2) Caso su implementación contenga *bugs*, *code smells* y *vulnerabilities*, corregir con la ayuda de la extensión/plugin SonarLint en su IDE.
- 3) Describir las prácticas o estilos relacionados a las convenciones de codificación usadas en un documento (README de repositorio Github o PDF adjuntado en esta práctica): Práctica + Fragmento de Código.

- 4) Subir (*Upload* o *Commit + Push* o *Pull Request*) sus cambios a la rama de desarrollo en *GitHub*.
 - a. Solicitar al propietario del repositorio del equipo para que este último incorpore los cambios (vía *Merge* o *Pull Request*), cuando sea necesario.
- 5) Actualizar el avance del proyecto por **Trello**:
 - a) Iteración actual:
 - Etapas: mover **tarjetas** que están siendo implementadas,
 - Responsables: Por implementar, y plazos,
 - *Features*:
 - **Checklist**: ~~“Escenarios de Prueba”~~: adicionar los ~~“escenarios”~~ a ser implementados y probados.
 - *Epics* o Historias asociadas a una *feature* de la iteración actual:
 - **Checklist**: “Tareas de Implementación”: adicionar tareas de implementación, por ejemplo, como “Implementar Controlador (REST); Estudiar Java y JPA y Hibernate; Mapear Dominio con ORM; Implementar Repositorio, ...

3. Recursos de Software:

- **Convenciones de Codificación** (*Coding Conventions or standards*) : Depende del Lenguaje de Programación escogido.
- **IDE**: Eclipse, VS Code, MS VS, ...
- **SonarLint** como extensión de su **IDE**: <https://www.sonarsource.com/products/sonarlint/>
 - Instalar SonarLint.
 - Reiniciar su IDE.
 - Construir el proyecto.

- **Instalacion y uso de SonarLint en diferentes IDEs:**
 - **SonarLint for IntelliJ and other JetBrains IDEs:** <https://www.youtube.com/watch?v=6Bv1wmj0jZI>
 - **SonarLint for VS Code:** <https://www.youtube.com/watch?v=m8sAdYCIWhY>
 - **SonarLint for Eclipse:** <https://www.youtube.com/watch?v=DJkNI6Q5I5A>
 - **SonarLint for Microsoft Visual Studio:** <https://www.youtube.com/watch?v=nASTGaxYXOo>
 - **SonarLint for PyCharm:** https://www.youtube.com/watch?v=pAz9O_N1Vs8

Entregable:

- Repositorio **GitHub** del equipo [cuya estructura debe estar organizada de acuerdo a las recomendaciones de *DDD*, el lenguaje de programación y los *frameworks* elegidos. El README debe presentar la siguiente información:
 - Propósito
 - Funcionalidades: Funcionalidades de Alto Nivel (Diagrama de Casos de Uso UML) y Prototipo (o GUI)
 - Modelo de Dominio: Diagrama de Clases + Módulos.
 - Vista General de Arquitectura: Diagrama de Paquetes + Clases
 - Convenciones de Codificación: Práctica + Fragmento de Código
- Tablero **Kanban/Scrum** de Proyecto basado en la plantilla "**User Story mapping**" en **Trello** u otra herramienta similar.

Referencias

- E. Evans. Domain-Driven Design: Tackling Complexity in the Heart of Software. 2003
- Valente, M. T. (2020). Engenharia de Software Moderna (Livro Digital). Disponible em: <https://softengbook.org/>
- Vernon, V. (2016). Domain-driven design distilled. Addison-Wesley Professional.
- Marinescu, F., & Avram, A. (2007). Domain-driven design Quickly. <https://www.infoq.com/minibooks/domain-driven-design-quickly/>
- Valente, M. T. (2020). [Engenharia de Software Moderna](#) (Livro Digital).
- M Richards. 2015. [Software architecture patterns](#).
- Mark Richards & Neal Ford. (2023). Fundamentals of software architecture: an engineering approach. O'Reilly.
- Garlan, David, and Mary Shaw. "An introduction to software architecture." Advances in software engineering and knowledge engineering. 1993. 1-39.

Material de Aula

- **Convenciones del Lenguaje de Programación:**
 - **JAVA:** <https://www.oracle.com/java/technologies/javase/codeconventions-contents.html>
 - <https://www.oracle.com/technetwork/java/codeconventions-150003.pdf>
 - **C#:** <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names>
 - <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>
 - **Python:** <https://peps.python.org/pep-0008/#naming-conventions>
 - <https://realpython.com/python-pep8/#naming-styles>
 - **Principios KISS, DRY y YAGNI:** <https://medium.com/@hlfddev/kiss-dry-solid-yagni-a-simple-guide-to-some-principles-of-software-engineering-and-clean-code-05e60233c79f>
- **Guías de Implementación de DDD y CLEAN ARCHITECTURE CON MVC y ORM (Ver diapos. DDD y otras fuentes):**
 - **Java:** <https://www.baeldung.com/spring-boot-start>
 - <https://www.baeldung.com/hexagonal-architecture-ddd-spring>
 - <https://www.digitalocean.com/community/tutorials/spring-mvc-hibernate-mysql-integration-crud-example-tutorial>

- <https://www.geeksforgeeks.org/spring-mvc-and-hibernate-crud-example/>
- **C#:** <https://learn.microsoft.com/en-us/aspnet/mvc/overview/getting-started/>
 - <https://cantinhode.net/blogs/community-cantinho-de-net/implementing-domain-driven-design-in-asp-net-mvc>
 - <https://banisd.wordpress.com/software-thoughts/ddd-mvc-entity-framework/>
- **Python:** <https://realpython.com/flask-connexion-rest-api/>
 - <https://realpython.com/flask-project/>
 - <https://flask.palletsprojects.com/en/3.0.x/>
 - <https://www.digitalocean.com/community/tutorials/how-to-structure-a-large-flask-application-with-flask-blueprints-and-flask-sqlalchemy>